

Nolte Geoservices UXO Target Platform

ETL Operator, Maintenance & Scheduling Architecture Guide

Document Version: 2.4	Target Environment: Production / Staging	Last Updated: 2026-09-29	Status: Fully Operational
-----------------------	--	--------------------------	---------------------------

Overview

The ETL pipeline coordinates data between source geodata tables (managed in QGIS/GIS workflows) and the live production application (`public.anomalies`, `public.feedback`).

This guide covers operational procedures, configuration validation, Web UI approvals, project-scoped runs, webhook notifications, and Slowly Changing Dimensions (SCD Type 2) audit history tracking.

1. Quick Reference & Core Commands

Action	Command / Location	Privileges Required
Trigger Full Run	Web UI: Database icon -> Run Sync Now CLI: <code>docker compose --profile etl run --rm etl run --force</code>	Admin user (UI) / <code>etl_pipeline</code> (CLI)
Trigger Single Project	Web UI: Select project -> Run Sync Now CLI: <code>docker compose --profile etl run --rm etl run --force --project 11-26-5151</code>	Admin user (UI) / <code>etl_pipeline</code> (CLI)
Start Cron Orchestrator	CLI: <code>docker compose --profile etl run --rm etl cron --schedule "/15 *"</code> Daemon: Container running with <code>ETL_CRON_SCHEDULE</code>	<code>etl_pipeline</code>
Validate Configuration	<code>docker compose --profile etl run --rm etl validate-config</code>	Read-only
Check Pipeline Health	Web UI: Top KPI Ribbon API: <code>GET /api/etl/health</code>	Read-only
Review Approvals (Web)	Web UI: Click Database icon in navigation rail	Admin user (<code>etl_approver</code>)
Review Approvals (CLI)	<code>docker compose --profile etl run --rm etl-approve status --approver</code>	<code>etl_approver</code>
Approve / Reject Staged	<code>docker compose --profile etl run --rm etl-approve approve-change <ids...></code> <code>docker compose --profile etl run --rm etl-approve reject-change <ids...></code>	<code>etl_approver</code>
Approve / Reject Shift	<code>docker compose --profile etl run --rm etl-approve approve-correction <pair_id></code> <code>docker compose --profile etl run --rm etl-approve reject-correction <pair_id></code>	<code>etl_approver</code>
Inspect Audit History	Web UI: Database -> Audit History tab API: <code>GET /api/etl/history?limit=100</code>	Admin user (<code>etl_approver</code>)

2. Configuration & Validation

The pipeline configuration is stored in `etl/config/projects.yml`.

Configuration Linter

Before deploying or running after editing `projects.yml`, run the config linter:

```
docker compose --profile etl run --rm etl validate-config
```

The linter validates:

- **Global Settings:** Validates `correction_radius_m` is positive.
- **Projects:** Verifies project IDs follow the `YY-NN-NNNN` pattern, schemas exist, and SRID is a valid positive integer (e.g., 25832, 25833).

- **Sources:** Verifies table names, instruments (**magnetic**, **georadar**), and coordinate column definitions (**easting**, **northing**).
- **Data Types & Bounds:** Verifies coordinate columns in Postgres are numeric/float types and fall within valid UTM boundaries.

3. Web Approver & Runner Interface

Admins can manage the entire ETL lifecycle from the web application without using command-line tools:

1. **Access:** Click the **Database** icon in the left-hand navigation rail. A badge shows pending approvals in real-time.
2. **Top Health Ribbon:** Displays real-time **Pipeline Health** (Healthy/Degraded/Unhealthy), database connection ping latency (ms), freshness SLA status, and counts of pending staged changes and coordinate shifts.
3. **Execution Bar:**
 - **Project Selector:** Run across all projects or select a specific project (**11-26-5151**, **11-24-2736**).
 - **Force Sync Toggle:** Bypasses the fingerprint cache to immediately apply decisions.
 - **Run Sync Now:** Triggers an execution in the background and refreshes the panel once finished.
4. **Staged Changes Tab:**
 - Shows attribute differences (**category**, **layer**, **evaluated_depth**, **instrument**).
 - Select individual rows or use "Select All" for bulk approve/reject.
5. **Coordinate Corrections Tab:**
 - Compares existing coordinates with new source coordinates.
 - Displays spatial shift distance in meters and rule match rationale (**source_key** vs spatial proximity).
6. **Runs & Health Tab:**
 - Inspect the last 25 pipeline executions with interactive SVG charts for duration trends (latency), ingestion volume vs staged changes, outcome distribution, and run status filters (**All**, **Successful**, **Skipped**, **Failed**).
7. **Audit History (SCD Type 2) Tab:**
 - Inspect full historical timeline of all changes made to targets.
 - Search by Target ID, VM Number, Project, or Change Reason.

4. Slowly Changing Dimensions (SCD Type 2) Audit History

Target history is maintained in **public.anomaly_history** through the PostgreSQL trigger **trigger_anomaly_scd_audit**.

Lifecycle:

1. **Creation:** When a new anomaly is inserted, an initial record is created with **is_current = true**, **valid_from = now()**, and **valid_to = NULL**.
2. **Modifications:** When attributes or coordinates change:
 - The prior active record is closed (**valid_to = now()**, **is_current = false**).
 - A new current record is inserted (**valid_from = now()**, **valid_to = NULL**, **is_current = true**).
 - The change reason is tagged automatically (**etl_change**, **etl_correction**, **field_sync**, **initial**).
 - If applied via ETL approver, the associated **decision_id** is recorded.
3. **Integrity:** A partial unique index (**anomaly_id**) **WHERE is_current = true** enforces at most one current record per target at the database engine level.

5. Webhook Notifications

Set **ETL_ALERT_WEBHOOK_URL** in **.env** to enable automated alerts:

- **Slack / Teams / Discord / Generic HTTP:** Posts JSON notifications automatically.
- **Alert Events:**
 - **FAILED:** Triggered on unhandled errors or safety gate failures with traceback.
 - **PENDING:** Triggered when a run holds back changes or shifts awaiting human review.
 - **SUCCESS:** Triggered on successful merges summarizing written and staged row counts.

6. Security & Privilege Separation

- **etl_pipeline**: Runs the pipeline. Possesses **INSERT** and **SELECT** on targets, but **never UPDATE** or **DELETE** on **public.anomalies**. It executes changes exclusively via PostgreSQL **SECURITY DEFINER** functions in **etl_admin**.
- **etl_approver**: Used exclusively for reviewing and deciding staged changes and coordinate shifts. It cannot write directly to **public.anomalies**.
- All decisions require explicit human review before modifying production records.

7. CI/CD Automated Validation

A GitHub Actions pipeline ([.github/workflows/etl-ci.yml](#)) automatically validates changes on every push and pull request touching ETL or frontend components:

- **Config & Schema Linting**: Validates **projects.yml** syntax, positive integer SRIDs, and column requirements (**python runner/run.py validate-config**).
- **Automated Unit Tests**: Executes unit test suites verifying schema linter, webhook payloads, and spatial bounds.
- **Frontend Quality Gates**: Executes ESLint and compiles production Vite bundles (**npm run build**).

8. Structured JSON Logging & Rotation

The pipeline features dual-destination structured logging via **etl/runner/logger.py**:

- **Console (stdout)**: Formatted, human-readable execution output tagged with run and project IDs (**[2026-09-29 11:00:00] [INFO] [run:72] [proj:11-26-5151] ...**).
- **Rotating JSON Lines File (etl/logs/etl.jsonl)**: Machine-readable JSON records including UTC timestamps, severity levels, contextual run IDs, project IDs, and exception stack traces. Rotates automatically at 10 MB with 5 backups.

9. Pipeline Health & Staleness Monitoring (/api/etl/health)

An operational health endpoint is available at **GET /api/etl/health** for monitoring systems (Datadog, Prometheus, UptimeRobot, ALB):

- **Database Latency**: Measures database connection latency in milliseconds.
- **Staleness SLA**: Compares elapsed time since the last successful merge against **ETL_MAX_STALENESS_HOURS** (default 24h).
- **Health Classifications**:
 - **healthy**: Database reachable, last run succeeded or skipped, freshness within SLA.
 - **degraded**: Pipeline is stale or pending approvals exceed thresholds.
 - **unhealthy**: Database connection failed or last execution resulted in a gate failure.
- **Approval Metrics**: Reports real-time counts of staged changes and correction pairs awaiting human action.

10. Execution Metrics Dashboard & Visual Trend Charts

The Web UI **ETL Pipeline Panel** (**Runs & Health** tab) provides real-time visual observability over execution performance:

- **KPI Summary Strip**:
 - **Avg Duration**: Mean execution latency across runs.
 - **Success Rate**: Percentage of successful vs failed runs.
 - **Total Merged**: Cumulative rows inserted/updated into **public.anomalies**.
 - **Total Runs**: Evaluated run window with skipped count.
- **Interactive SVG Charts**:
 - **Duration Trend (Latency)**: Chronological run durations (seconds) color-coded by status (Green = OK, Amber = Skipped, Red = Failed) with interactive hover tooltips.
 - **Ingestion Volume & Staged Changes**: Dual bars displaying anomalies written vs. attribute changes held in staging.
 - **Outcome Distribution**: Segmented progress bar illustrating proportion of Successful, Skipped, and Failed executions.
 - **Run Filtering**: Filter historical runs dynamically by **All**, **Successful**, **Skipped**, or **Failed** to isolate and diagnose anomalies or execution errors.

11. Pipeline Orchestrator & Scheduling Architecture

The pipeline can be scheduled using multiple deployment strategies depending on infrastructure requirements:

Option A: Built-in Structured Cron Orchestrator (Recommended for Containers)

The runner includes a native 5-part cron evaluator with tick alignment (firing precisely on clock boundaries without drift) and graceful signal handling (**SIGTERM/SIGINT**):

```
# Run on standard cron schedule (e.g., every 15 minutes)
python runner/run.py cron --schedule "*/15 * * * *"

# Or via Docker Compose environment variable:
ETL_CRON_SCHEDULE="*/15 * * * *"
```

Option B: Linux Host Crontab / Systemd Timer

Run isolated ephemeral containers on host cron without running continuous daemon containers:

```
# /etc/cron.d/etl-pipeline: run every hour at minute 0
0 * * * * root cd /opt/feedbackapp && docker compose run --rm etl run >> /var/log/etl.log 2>&1
```

Option C: Native PostgreSQL pg_cron

For managed PostgreSQL environments with the **pg_cron** extension pre-loaded (**shared_preload_libraries = 'pg_cron'**):

```
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- Trigger ETL runner container or webhook via pg_cron:
SELECT cron.schedule('etl_merge_tick', '*/15 * * * *',
  $$SELECT net.http_post(
    url='http://app:8000/api/etl/run-now',
    headers:='{ "Authorization": "Bearer ...", "Content-Type": "application/json" }'::jsonb
  );$$
);
```

12. Environment Variables Reference

Variable	Default	Purpose
ETL_CRON_SCHEDULE	/15 *	5-field cron expression for wall-clock aligned container scheduling.
ETL_INTERVAL_SECONDS	0 (off)	Fallback fixed sleep interval in seconds for legacy loop mode.
ETL_ALERT_WEBHOOK_URL	(unset)	Webhook endpoint for automated Teams / Slack / Discord alert notifications.
ETL_MAX_STALENESS_HOURS	24	SLA threshold in hours before /api/etl/health flags the pipeline as stale.
ETL_DB_HOST	localhost	PostgreSQL server hostname or container service name.
ETL_DB_PORT	5432	PostgreSQL listening port.
ETL_DB_NAME	nolte_geoservices	Target database name.
ETL_DB_USER	etl_pipeline	Least-privilege pipeline role credentials (cannot modify feedback or UPDATE targets directly).
ETL_DB_PASSWORD	(required)	Password for the etl_pipeline role.
ETL_APPROVER_USER	etl_approver	Least-privilege approver role credentials used by the review UI.
ETL_APPROVER_PASSWORD	(required)	Password for the etl_approver role.